The National Higher School of Artificial Intelligence
المدرسة الوطنية العليا للذكاء الاصطناعي

Course
« *Computer Vision*»

Sid-Ahmed Berrani

2025-2026

VII. The Hough transform

The Hough transform was originally introduced by **Paul Hough** in 1962:

U.S. patent 3,069,654 titled "Method and Means for Recognizing Complex Patterns".

The method was later generalized by **Richard Duda and Peter Hart** in 1972 to detect arbitrary shapes, particularly lines, in digital images:

> **Use** of the **Hough transformation** to **detect lines** and **curves** in pictures
> RO Duda, PE Hart - Communications of the ACM, 1972 - dl.acm.org
> … to the region of the 0-p plane near this **curve**. If we **find** a cell with count k near this **curve**, then we are assured that k figure points lie on a **line** passing (nearly) through the point (x0, y0). …
> ☆ Enregistrer Citer Cité 10811 fois Autres articles Les 7 versions

VII. The Hough transform

Definition and goal:

- The Hough Transform is a **feature extraction technique** used in image analysis, computer vision, and digital image processing.
- Its main goal is to detect **simple geometric shapes** (such as lines, circles, or ellipses) in an image.

VII. The Hough transform

Advantages over existing methods:

- **Robust to noise and gaps:** Unlike edge-following algorithms, the Hough Transform can detect lines even if the line is partially broken or noisy.
- **Simple mathematical formulation** for complex pattern recognition tasks.
- **Works in cluttered scenes**: Efficiently finds shapes even in the presence of many irrelevant edges.
- **Detects multiple instances** of the same shape in a single pass.

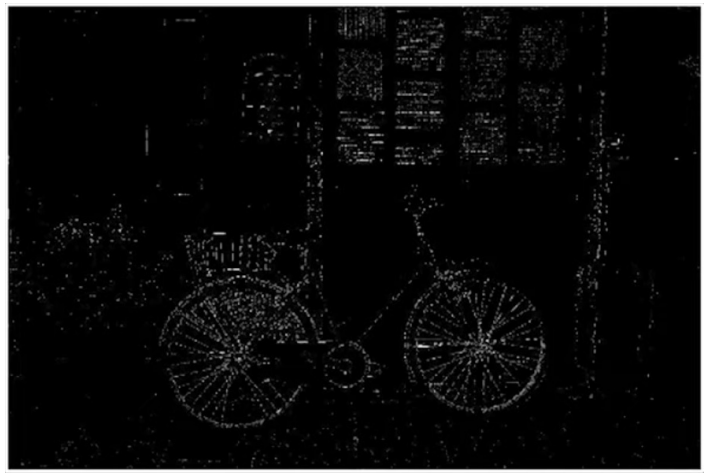
VII. The Hough transform

Applications:

- **Lane detection** in autonomous driving systems (detecting road boundaries or lanes).
- **Object recognition** where objects are approximated by geometric shapes.
- **Medical image analysis** (e.g., detecting circular features like tumors or bones).
- **Robotics** for identifying structured environments.
- **Industrial inspection** to detect regular shapes in manufactured parts.
- **Document image analysis** to detect lines, text baselines, or page segmentation.
- …

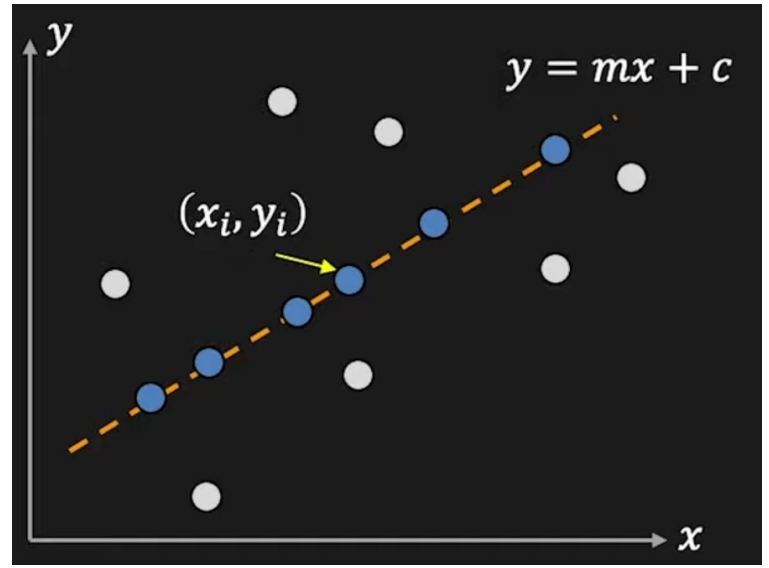
VII. The Hough transform

Challenges:



- Which data to fit to?
- Only part of the model is visible: data is incomplete
- Noise.

VII. The Hough transform



Let's start with the straight line…

Edges: points (x_i, y_i)

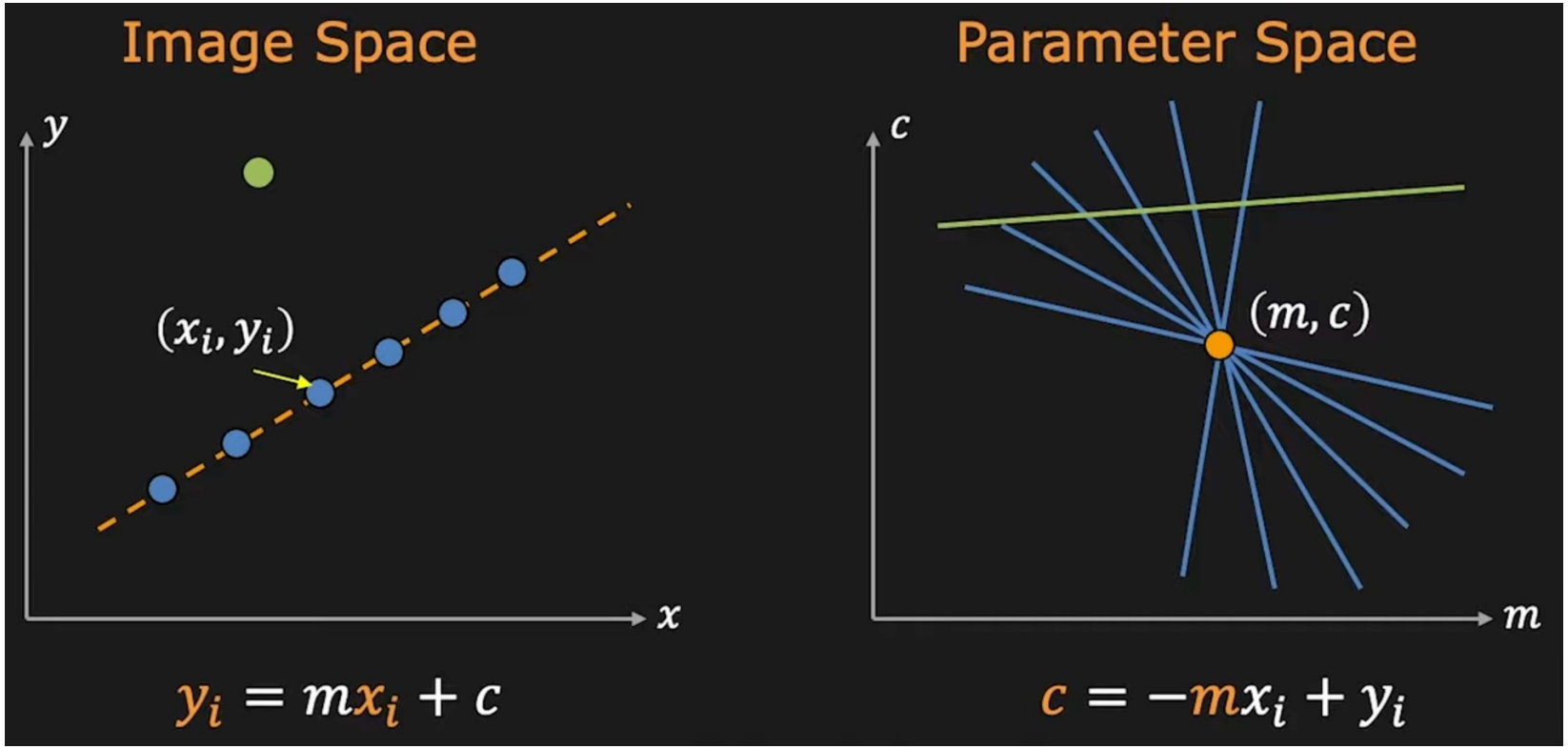
Task: detect the line $\mathrm{y} = mx + c$

If we consider a point (x_i, y_i):

$$y_i = mx_i + c \quad \Leftrightarrow \quad \mathrm{c} = -mx_i + y_i$$

VII. The Hough transform

The principle:



Point
Line

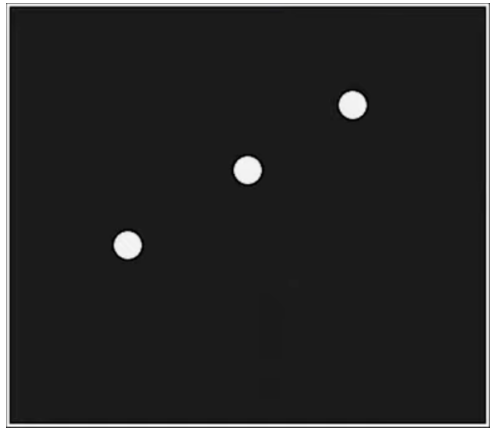
Line
Point

VII. The Hough transform

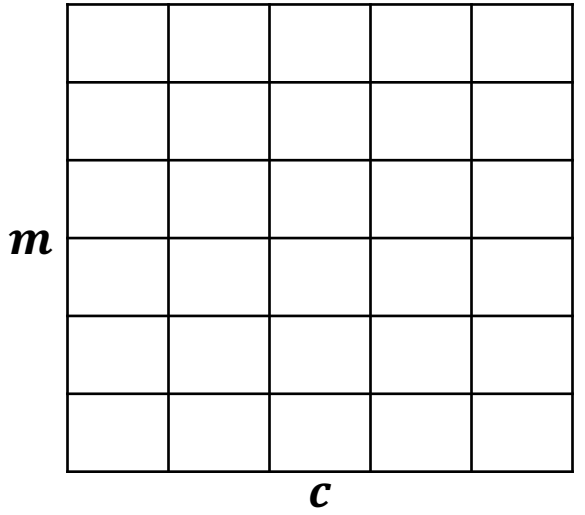
Line detection algorithm

1. Quantize parameter space (m, c)
2. Create an accumulator array $\mathrm{A}(m, c)$
3. Set $\mathrm{A}(m, c) = 0$ for all positions in the array

Image



$\mathbf{A}(\boldsymbol{m}, \boldsymbol{c})$



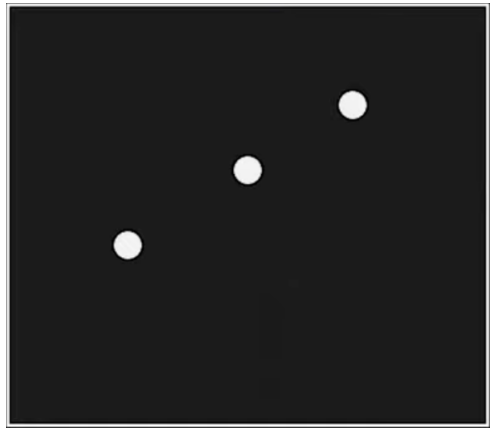
VII. The Hough transform

Line detection algorithm

1. Quantize parameter space (m, c)
2. Create an accumulator array $\mathrm{A}(m, c)$
3. Set $\mathrm{A}(m, c) = 0$ for all positions in the array
4. For each edge point (x_i, y_i):

   $\mathrm{A}(m, c)$ += 1 if (m, c) lies in the line $\mathrm{c} = -mx_i + y_i$

Image



0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

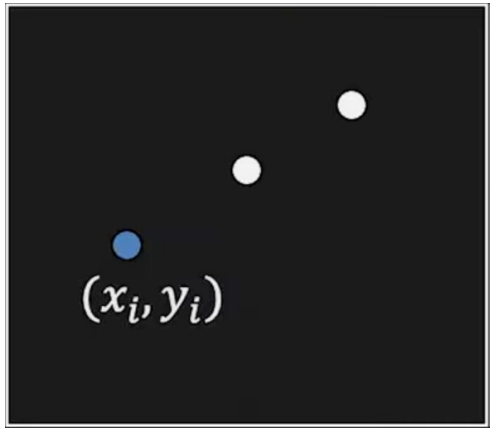
VII. The Hough transform

Line detection algorithm

1. Quantize parameter space (m, c)
2. Create an accumulator array $\mathrm{A}(m, c)$
3. Set $\mathrm{A}(m, c) = 0$ for all positions in the array
4. For each edge point (x_i, y_i):

   $\mathrm{A}(m, c)$ += 1 if (m, c) lies in the line $\mathrm{c} = -mx_i + y_i$

Image



$\mathbf{A}(\boldsymbol{m}, \boldsymbol{c})$

$\boldsymbol{m}$ \ $\boldsymbol{c}$				
1	0	0	0	0
0	**1**	0	0	0
0	0	**1**	0	0
0	0	0	**1**	0
0	0	0	0	**1**
0	0	0	0	0

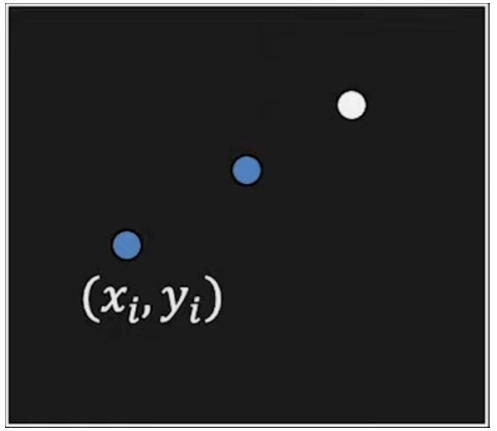
VII. The Hough transform

Line detection algorithm

1. Quantize parameter space (m, c)
2. Create an accumulator array $\mathrm{A}(m, c)$
3. Set $\mathrm{A}(m, c) = 0$ for all positions in the array
4. For each edge point (x_i, y_i):

   $\mathrm{A}(m, c)$ += 1 if (m, c) lies in the line $\mathrm{c} = -mx_i + y_i$

Image



$\mathbf{A}(\boldsymbol{m}, \boldsymbol{c})$

$\boldsymbol{m}$ \ $\boldsymbol{c}$				
1	0	0	**1**	0
0	**1**	0	**1**	0
0	0	**1**	**1**	0
0	0	0	**2**	0
0	0	0	**1**	**1**
0	0	0	**1**	0

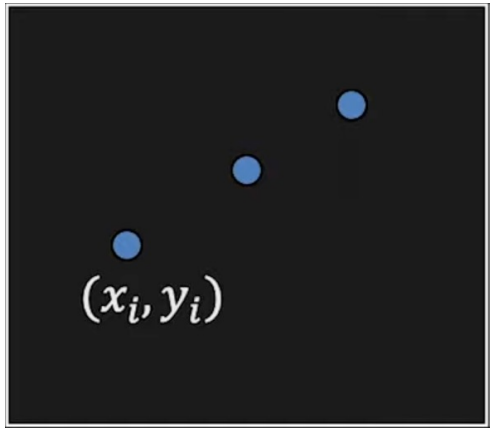
VII. The Hough transform

Line detection algorithm

1. Quantize parameter space (m, c)
2. Create an accumulator array $\mathrm{A}(m, c)$
3. Set $\mathrm{A}(m, c) = 0$ for all positions in the array
4. For each edge point (x_i, y_i):

   $\mathrm{A}(m, c)$ += 1 if (m, c) lies in the line $\mathrm{c} = -mx_i + y_i$

Image



$\mathbf{A}(\boldsymbol{m}, \boldsymbol{c})$

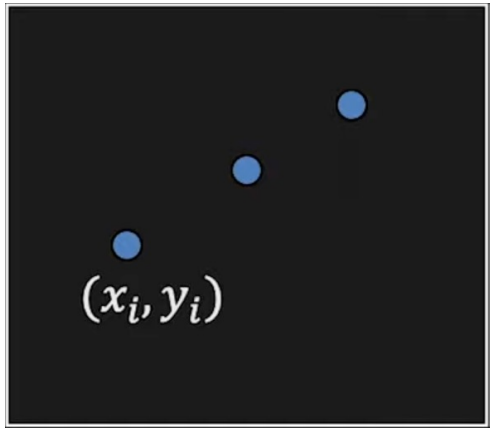
$\boldsymbol{m}$ \ $\boldsymbol{c}$				
1	0	0	**1**	0
0	**1**	0	**1**	0
0	0	**1**	**1**	0
1	**1**	**1**	**3**	**1**
0	0	0	**1**	**1**
0	0	0	**1**	0

VII. The Hough transform

Line detection algorithm

1. Quantize parameter space (m, c)
2. Create an accumulator array $\mathrm{A}(m, c)$
3. Set $\mathrm{A}(m, c) = 0$ for all positions in the array
4. For each edge point (x_i, y_i):
   $\mathrm{A}(m, c)$ += 1 if (m, c) lies in the line $c = -mx_i + y_i$
5. Find local maxima in $\mathrm{A}(m, c)$

Image



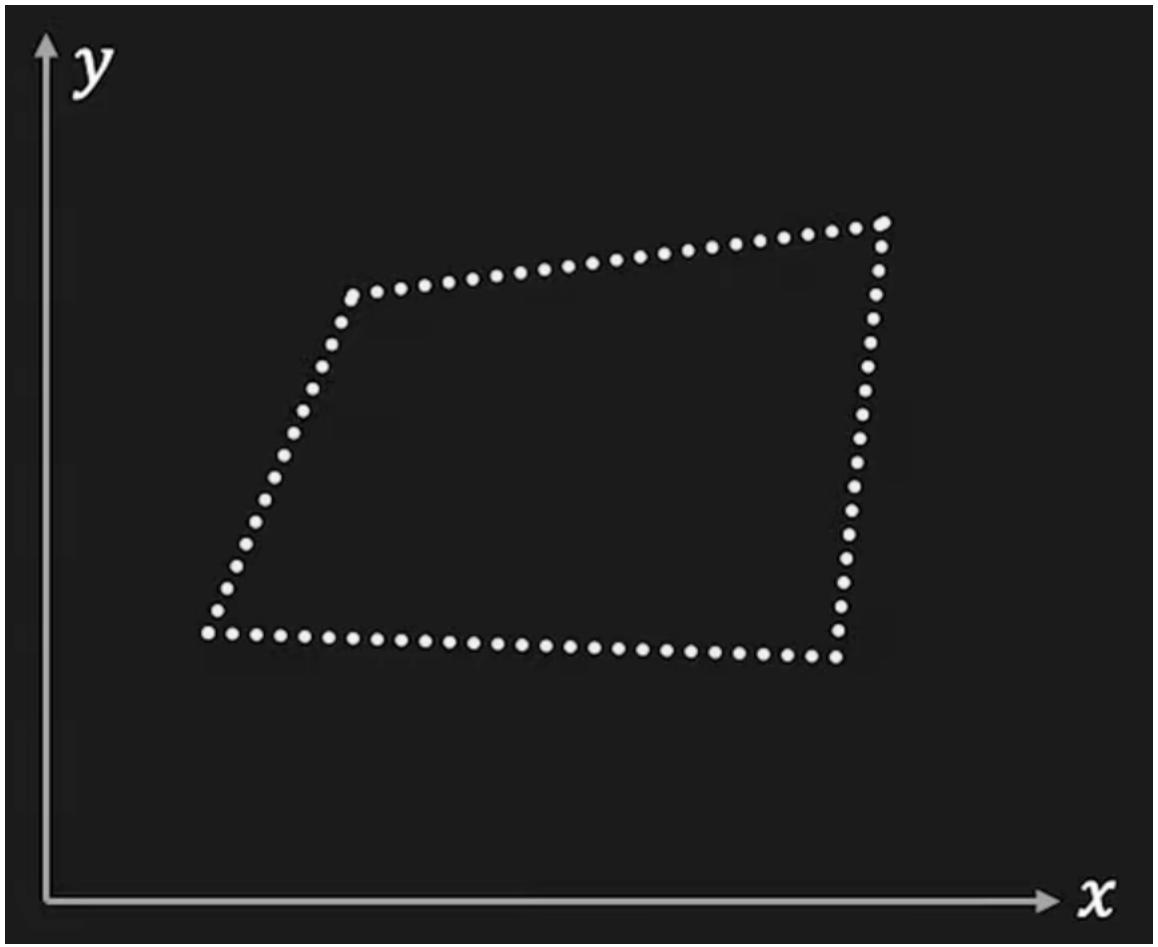
$\mathbf{A}(\boldsymbol{m}, \boldsymbol{c})$

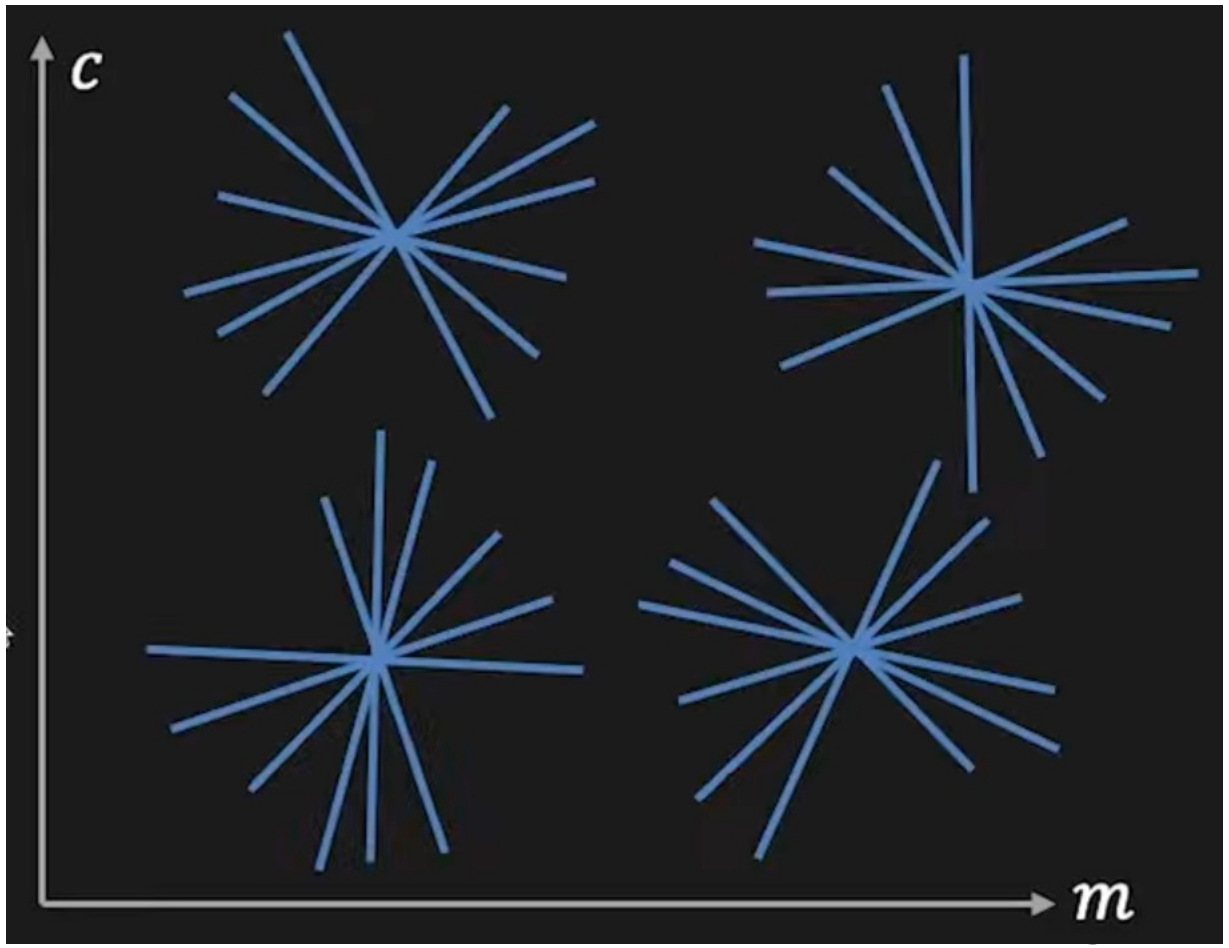
	1	0	0	**1**	0
	0	**1**	0	**1**	0
	0	0	**1**	**1**	0
$\boldsymbol{m}$	**1**	**1**	**1**	**3**	**1**
	0	0	0	**1**	**1**
	0	0	0	**1**	0

$\boldsymbol{c}$

VII. The Hough transform

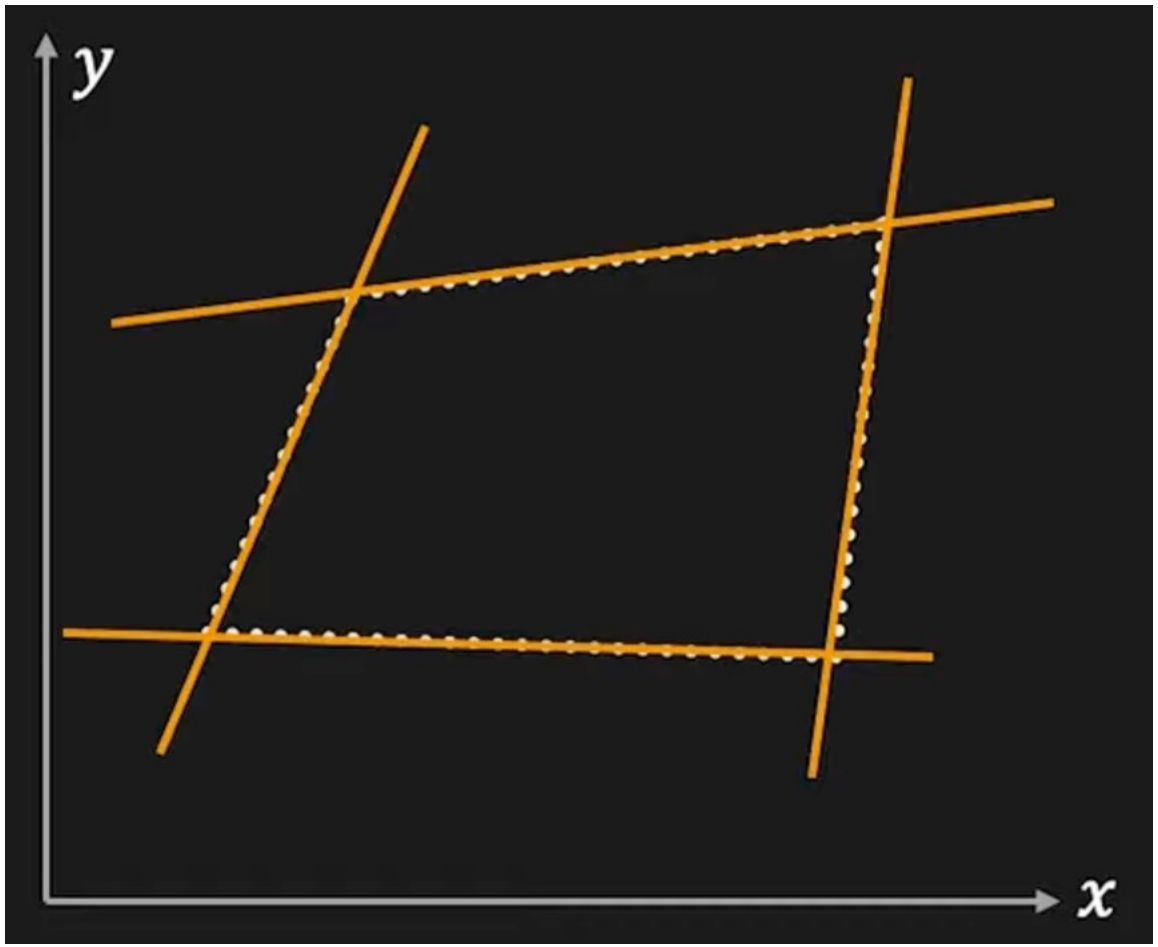
The case of multiple lines per image

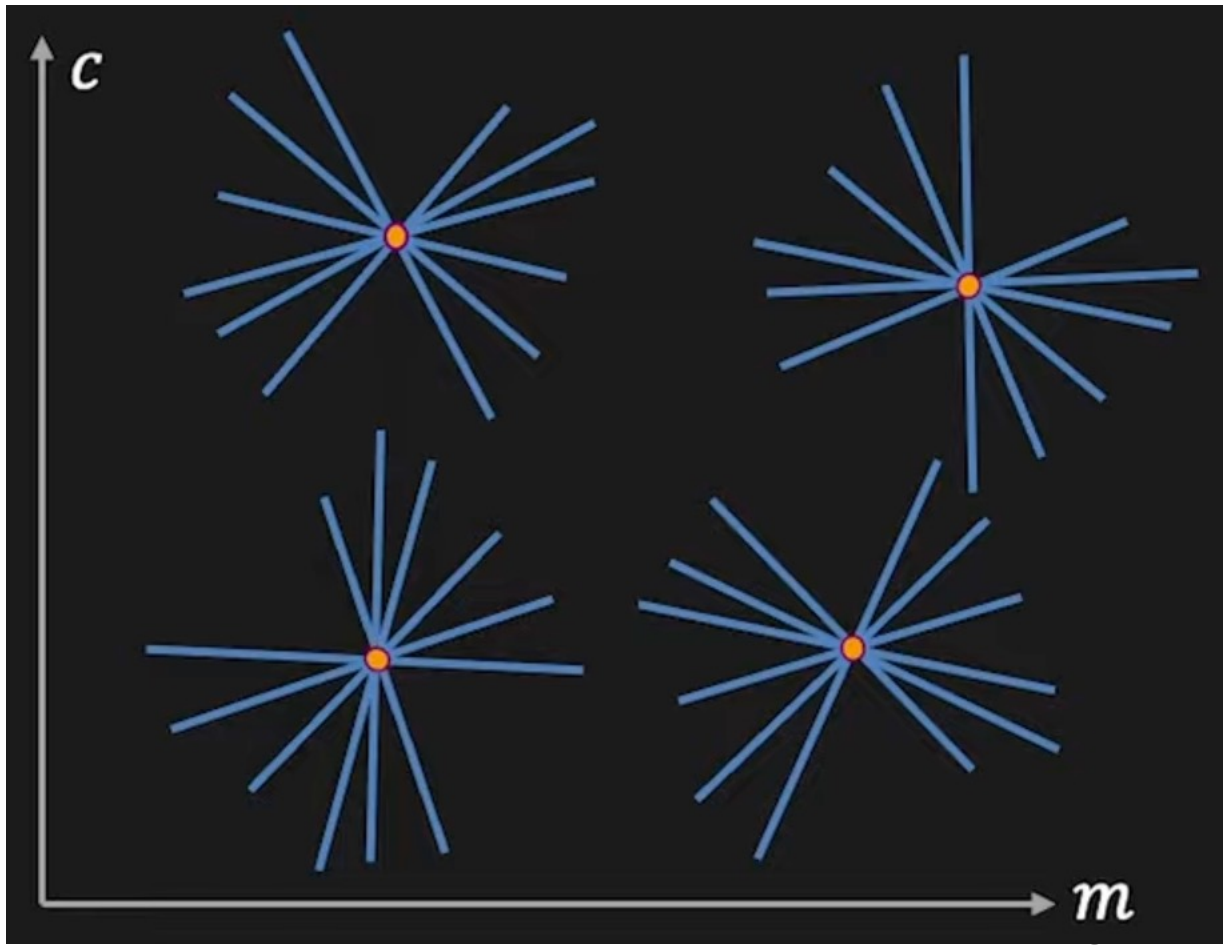




VII. The Hough transform

The case of multiple lines per image

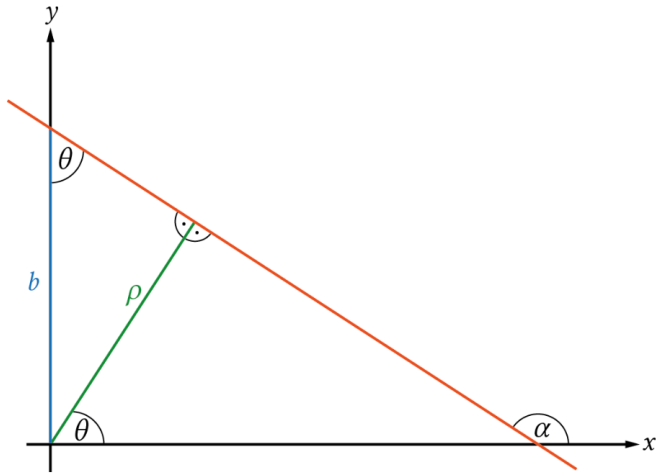




VII. The Hough transform

A parametrization problem

- The parameter m (the slope) varies between $-\infty$ and $+\infty$.
- A fine quantization => a very large accumulator.
- A huge amount of memory required
- A very costly line detection algorithm.



The solution:

- Instead of representing the line in the cartesian domain, use a **polar representation**.
- Work on parameters θ and ρ:

$$x\cos\theta + y\sin\theta = \rho$$

$$\pi \geq \theta \geq 0$$

ρ is finite

VII. The Hough transform

The algorithm:

- Initialize $\mathrm{A}(\rho, \theta) = 0$
- For each edge point $p_i(x_i, y_i)$ in the image

  For $\theta = 0$ to π

  $$\rho = x_i cos(\theta) + y_i sin(\theta)$$

  $$\mathrm{A}(\rho, \theta) \mathrel{+}= 1$$

- Find (ρ, θ) for which $\mathrm{A}(\rho, \theta)$ is maximum.
- The detected line is given by $\rho = x\ cos(\theta) + y\ sin(\theta)$

VII. The Hough transform

Possible extensions & optimizations

- Along with the edges, make use of the gradient direction

  => Reduces the need to iterate overall possible values of θ.

  The gradient direction at each edge point to directly infer the most likely orientation of the line passing through that point.

- Give more votes to strongest edges: Makes use of the gradient magnitude.

- Change the sampling of (ρ, θ) to trade-off resolution with computing time:

  High resolution -> Dispersion of votes (different line may be merged)

  Low resolution -> Cannot distinguish similar lines (noise may causes lines to be missed)

VII. The Hough transform

Possible extensions & optimizations

- How many lines do we have?

  Count the peaks in the accumulator array.

  Need a peak finding technique (non-maximal suppression for corner corner detection).

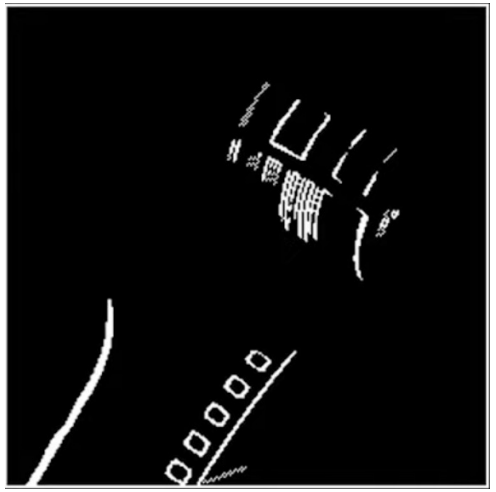
VII. The Hough transform

Examples

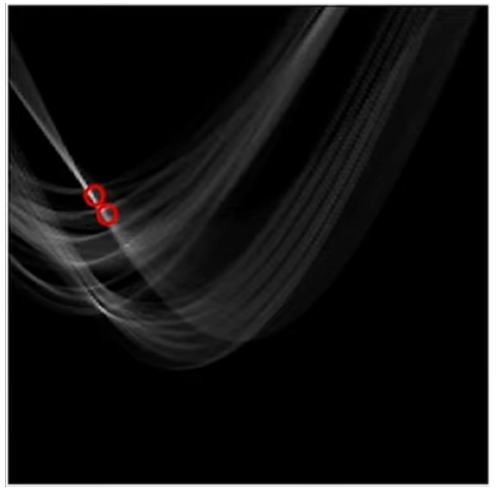
Original image

Gradient

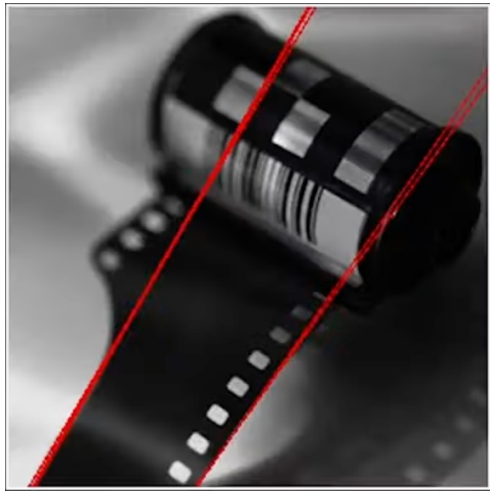
Edges



Hough Transform



Detected lines

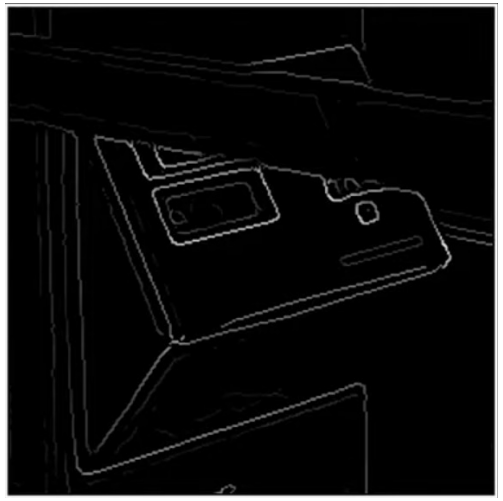


VII. The Hough transform

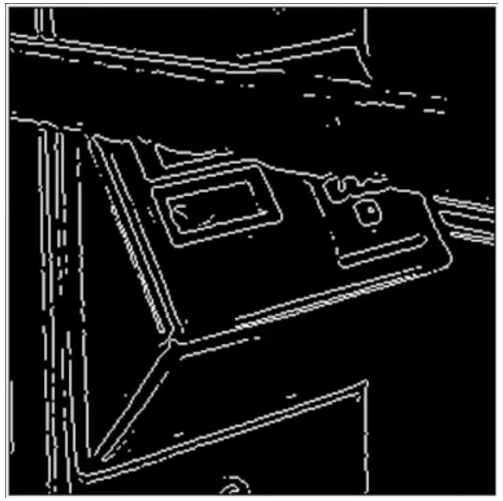
Examples

Original image

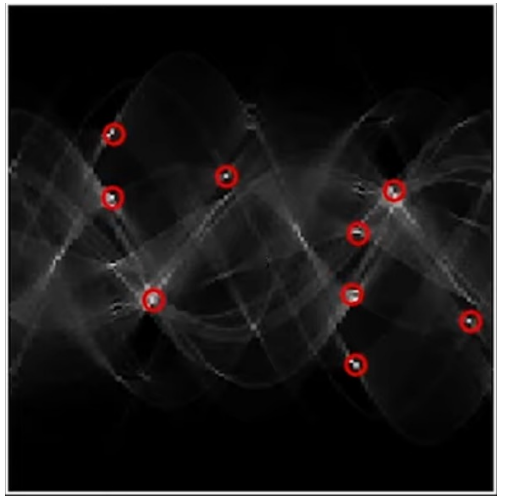
Gradient



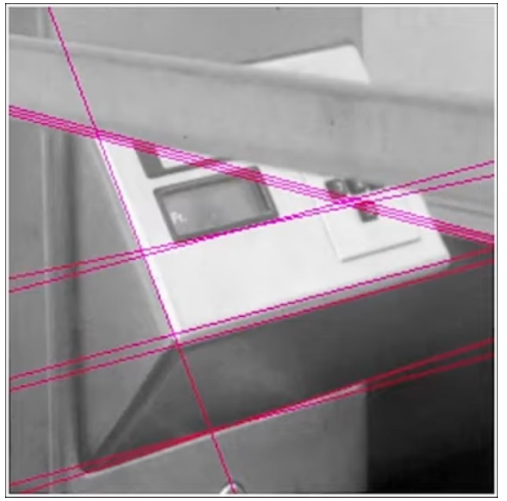
Edges



Hough Transform



Detected lines

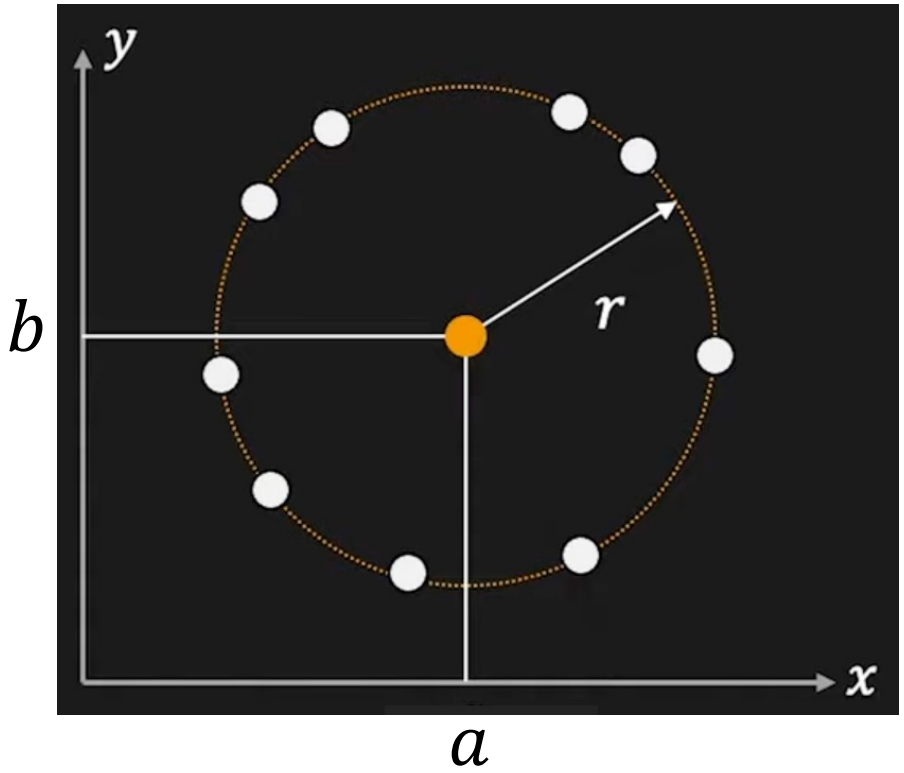


VII. The Hough transform

Application to circle detection

- $(x_i - a)^2 + (y_i - b)^2 = r^2$

$\Rightarrow$ Problem: find the three parameters a, b and r given a set of edge points.



VII. The Hough transform

Application to circle detection

- Let's make the problem simpler. We assume the radius r known.
- The accumulator array concerns only parameters a and b (the center of the circle).

$$(\boldsymbol{x_i} - a)^2 + (\boldsymbol{y_i} - b)^2 = r^2$$

$$(x_i - \boldsymbol{a})^2 + (\boldsymbol{y_i} - \boldsymbol{b})^2 = r^2$$

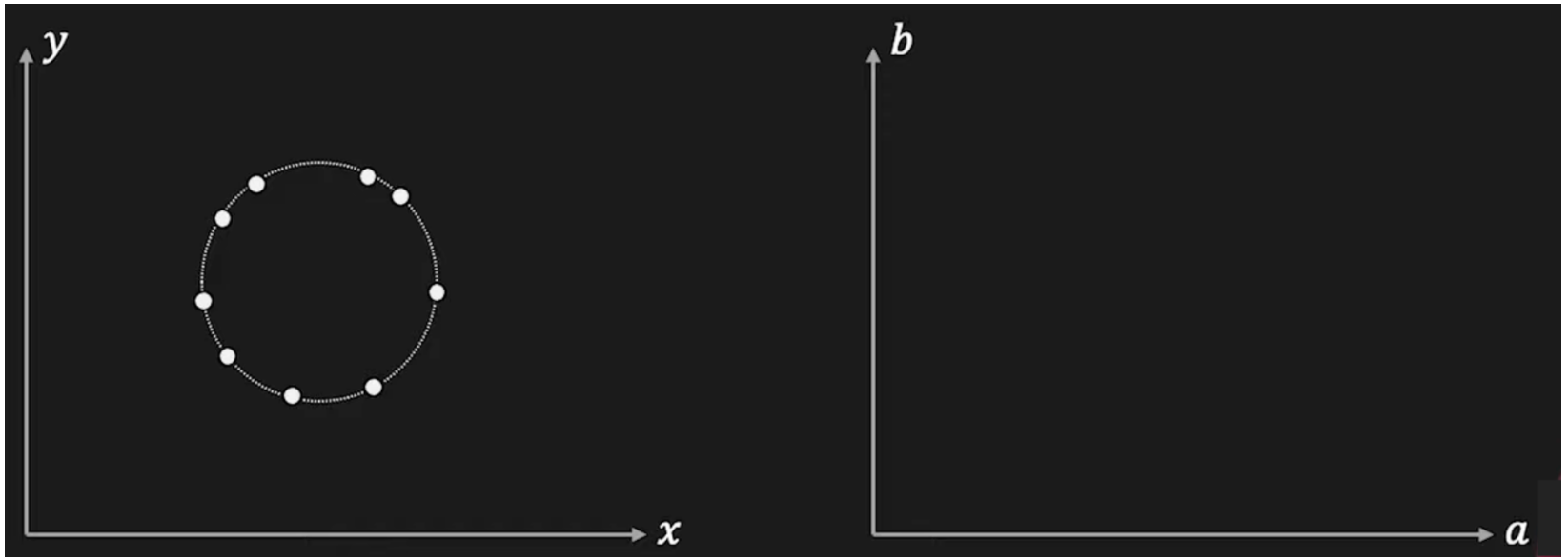


Image space

Parameter space

VII. The Hough transform

Application to circle detection

- Let's make the problem simpler. We assume the radius r known.
- The accumulator array concerns only parameters a and b (the center of the circle).

$$(\boldsymbol{x_i} - a)^2 + (\boldsymbol{y_i} - b)^2 = r^2$$

$$(x_i - \boldsymbol{a})^2 + (\boldsymbol{y_i} - \boldsymbol{b})^2 = r^2$$

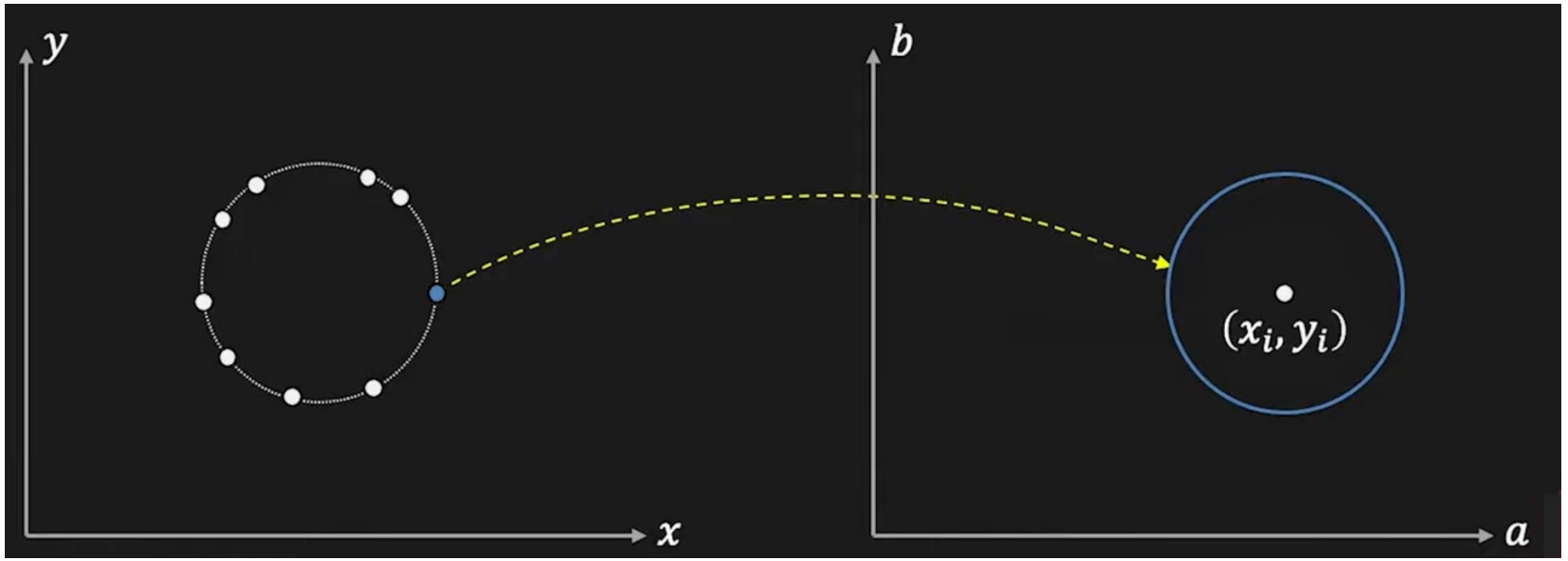


Image space **Parameter space**

VII. The Hough transform

Application to circle detection

- Let's make the problem simpler. We assume the radius r known.
- The accumulator array concerns only parameters a and b (the center of the circle).

$$(\boldsymbol{x_i} - a)^2 + (\boldsymbol{y_i} - b)^2 = r^2$$

$$(x_i - \boldsymbol{a})^2 + (\boldsymbol{y_i} - \boldsymbol{b})^2 = r^2$$

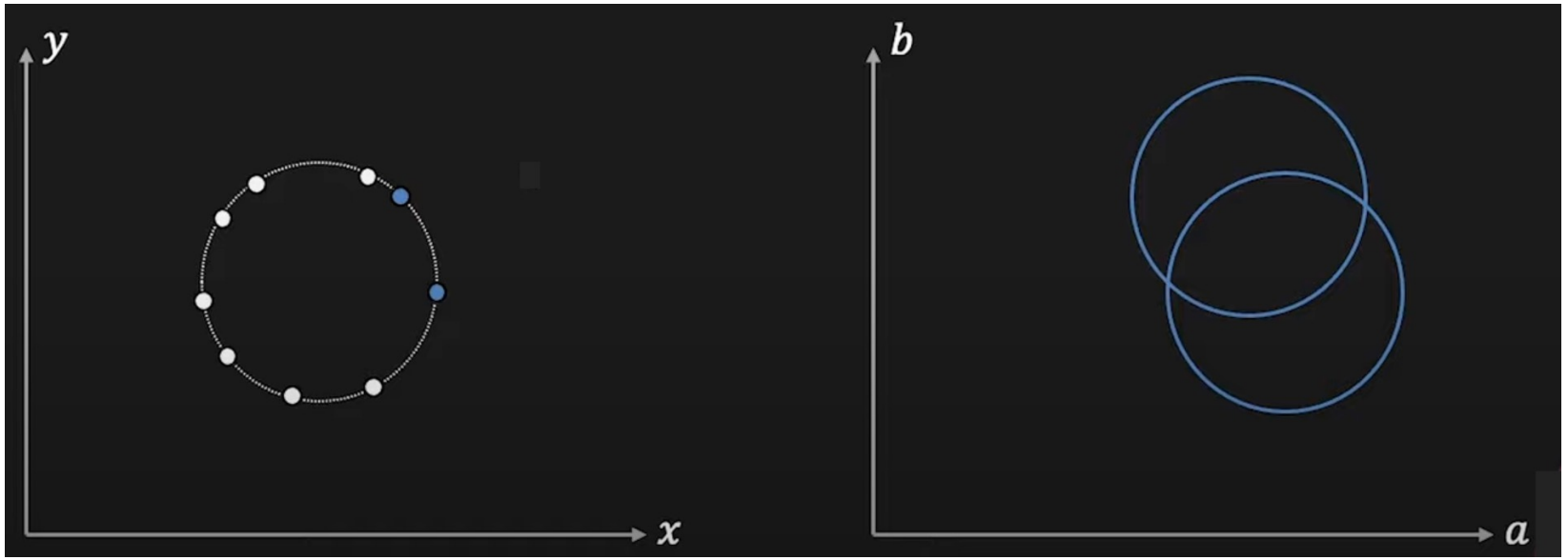


Image space

Parameter space

VII. The Hough transform

Application to circle detection

- Let's make the problem simpler. We assume the radius r known.
- The accumulator array concerns only parameters a and b (the center of the circle).

$$(\boldsymbol{x_i} - a)^2 + (\boldsymbol{y_i} - b)^2 = r^2$$

$$(x_i - \boldsymbol{a})^2 + (\boldsymbol{y_i} - \boldsymbol{b})^2 = r^2$$

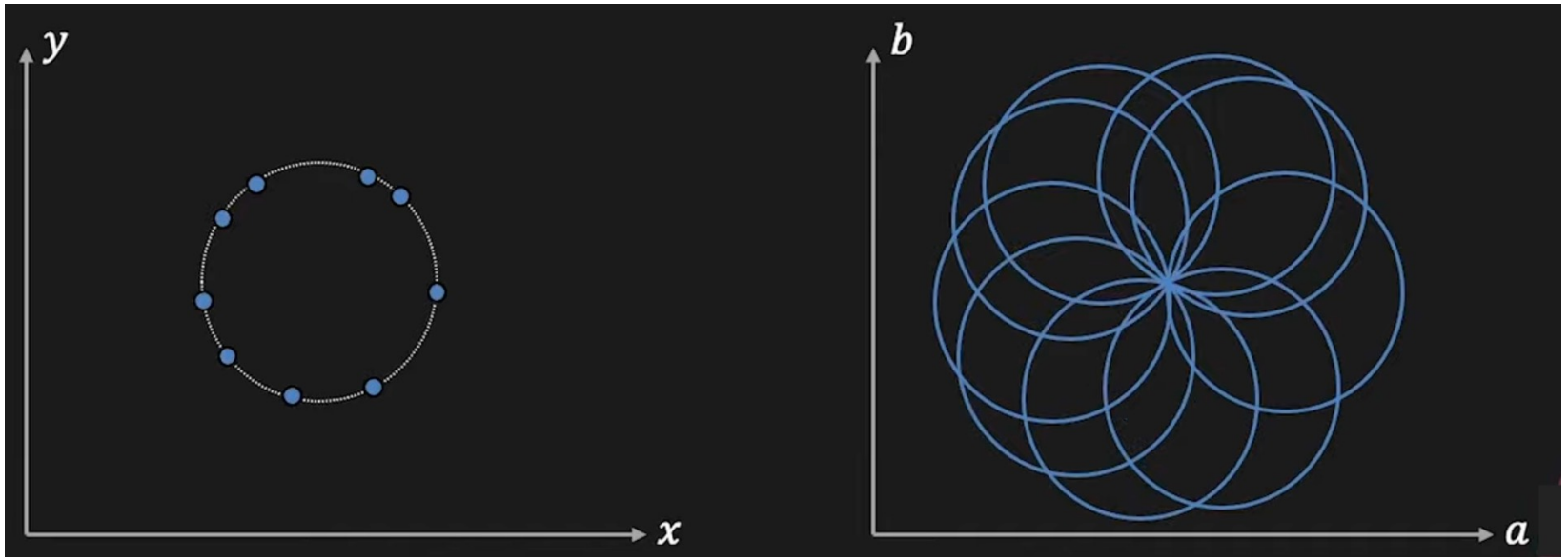


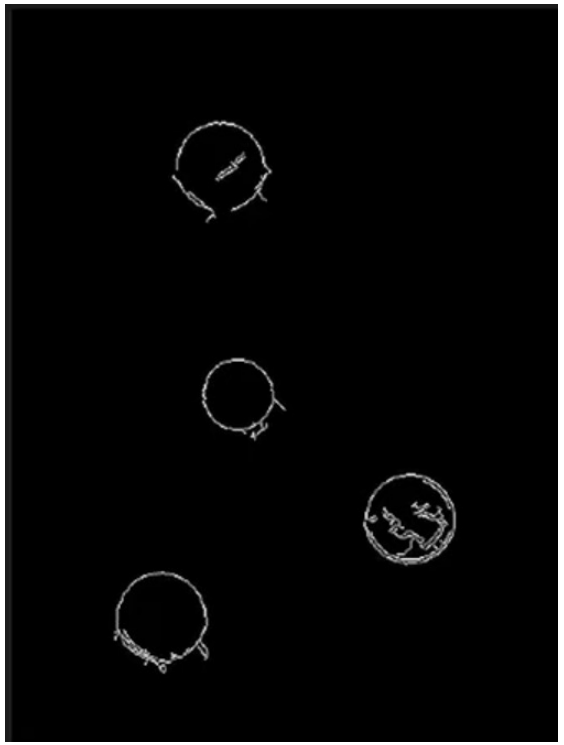
Image space

Parameter space

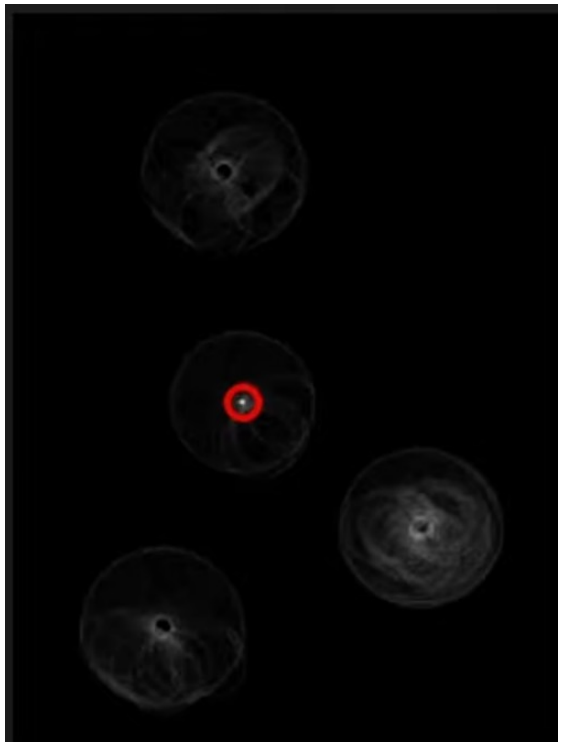
VII. The Hough transform

An example

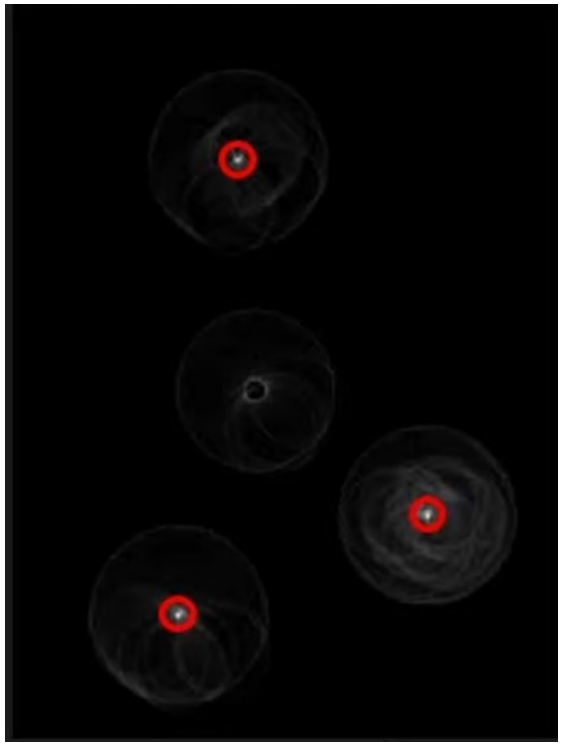
Original image



Edges



Hough Transform
$r = r_1$



Hough Transform
$r = r_2$

VII. The Hough transform

Application to circle detection

- Now, if the radius r is unknown.
- The accumulator array concerns parameters a, b and also the radius r.

$$(\boldsymbol{x_i} - a)^2 + (\boldsymbol{y_i} - b)^2 = r^2$$

$$(x_i - \boldsymbol{a})^2 + (\boldsymbol{y_i} - \boldsymbol{b})^2 = \boldsymbol{r}^2$$

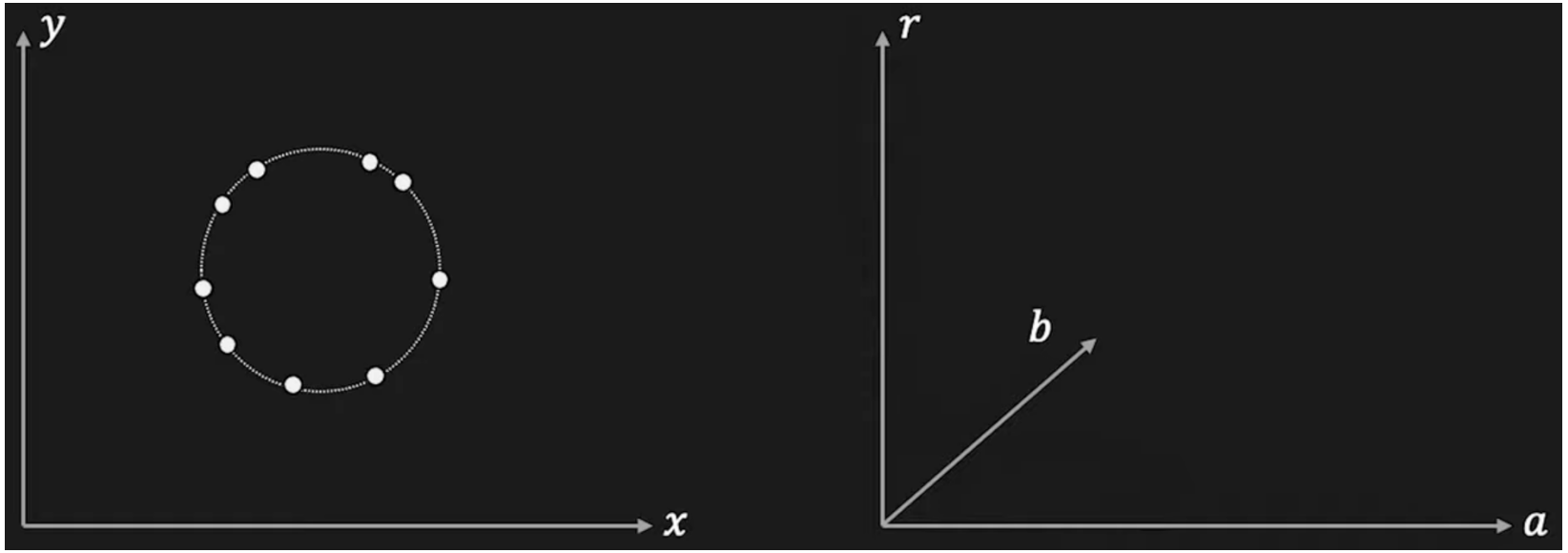


Image space

Parameter space

VII. The Hough transform

Application to circle detection

- Now, if the radius r is unknown.
- The accumulator array concerns parameters a, b and also the radius r.

$$(\boldsymbol{x_i} - a)^2 + (\boldsymbol{y_i} - b)^2 = r^2$$

$$(x_i - \boldsymbol{a})^2 + (\boldsymbol{y_i} - \boldsymbol{b})^2 = \boldsymbol{r}^2$$

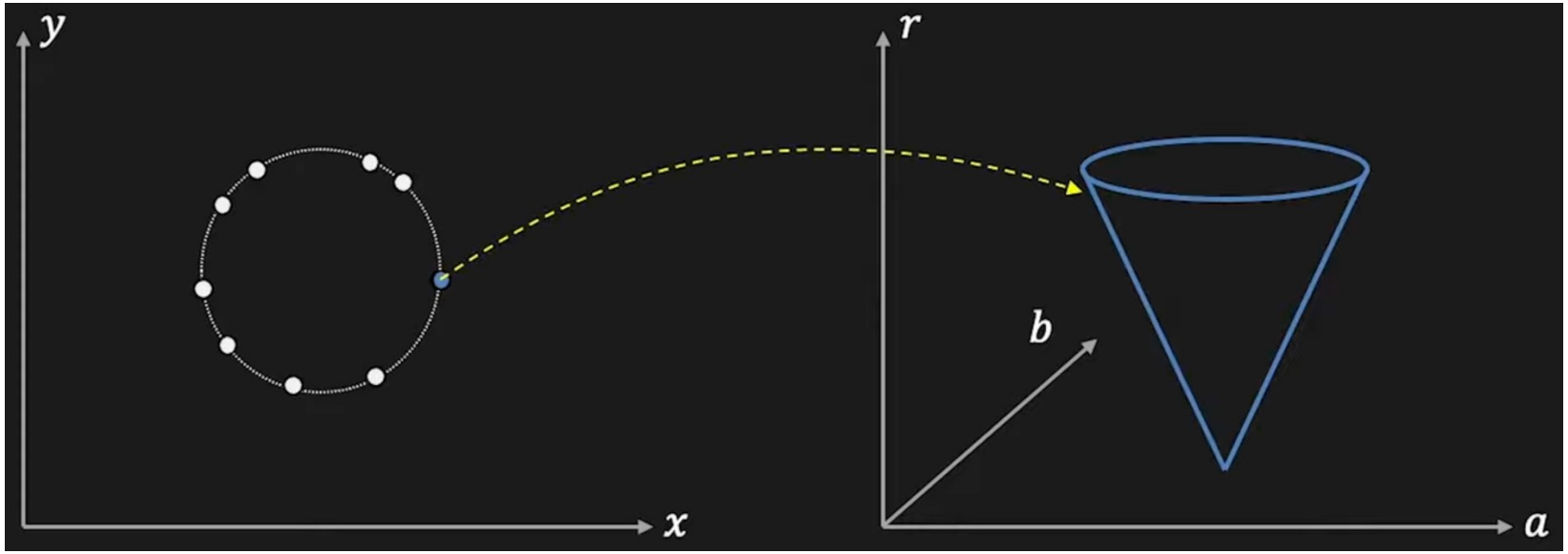


Image space

Parameter space

VIII. RANSAC: The Random Sample Consensus Algorithm

Ref.: Fischler and Bolles 1981.

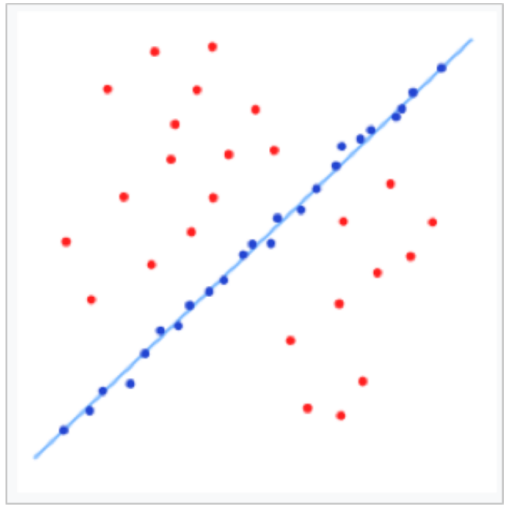
Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. Communications of the ACM, Volume 24, Issue 6. https://dl.acm.org/doi/10.1145/358669.358692

Principle

- A learning technique to estimate parameters of a model by random sampling of observed data.
- Given a dataset whose data elements contain both **inliers** and **outliers**, RANSAC uses a consensus scheme to find the optimal fitting result.

VIII. RANSAC: The Random Sample Consensus Algorithm

RANSAC



- It is a trial-and-error approach.
- Able to deal with high fractions of outliers in the data **(up to 50%)**.

VIII. RANSAC: The Random Sample Consensus Algorithm

RANSAC

- **Key idea:** Find the best partition of inliers/outliers in the data and estimate the model from the inlier subset.
- It the standard/recommended approach for fitting in the presence of outliers.

VIII. RANSAC: The Random Sample Consensus Algorithm

Assumptions

- Data consists of **inliers** (i.e., data whose distribution can be explained by some set of model parameters, though may be subject to noise) and **outliers** which are data that do not fit the model
- Given a (usually small) set of inliers, there exists a procedure which can estimate the parameters of a model that optimally explains or fits this data

For example, given a set of 2 points, we can compute a line model that optimally explains the set.

VIII. RANSAC: The Random Sample Consensus Algorithm

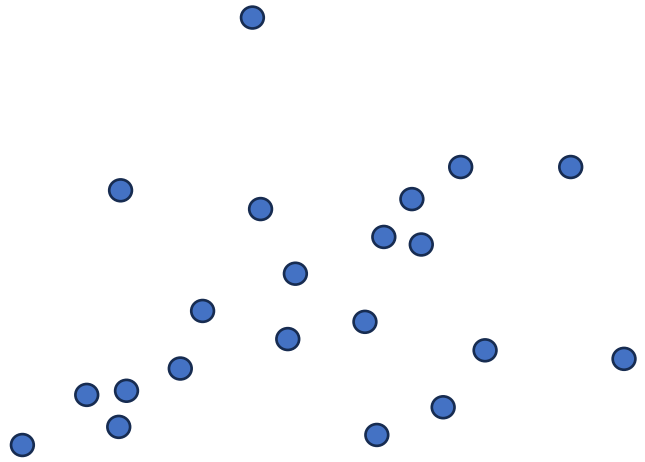
Algorithm

1. Select a random subset of the original data. Call this subset the hypothetical inliers.
2. Fit the model to the hypothetical inliers
3. Test all other data against the model and mark points either as inliers or outliers according to some loss function. The inliers are called “consensus set”
4. Return to step 1 until a predefined number of iterations **N** is reached
5. The model that produced the largest consensus set is returned

VIII. RANSAC: The Random Sample Consensus Algorithm

Algorithm

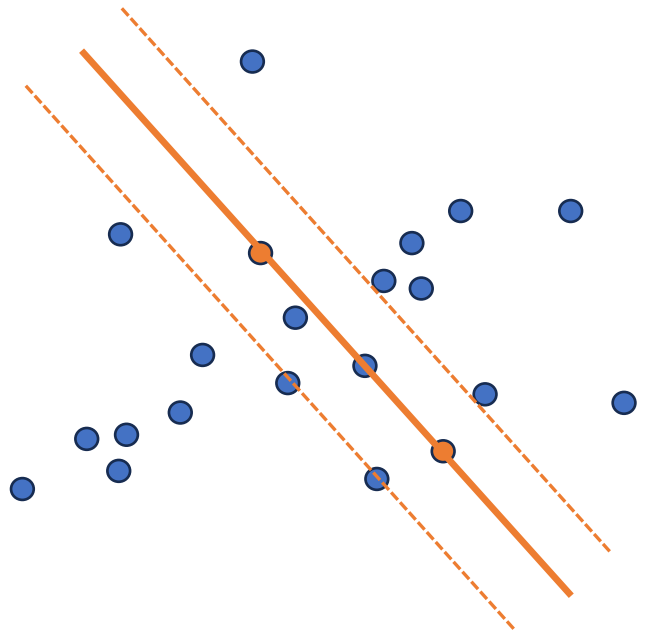
1. Select a random subset of the original data. Call this subset the hypothetical inliers.
2. Fit the model to the hypothetical inliers
3. Test all other data against the model and mark points either as inliers or outliers according to some loss function. The inliers are called “consensus set”
4. Return to step 1 until a predefined number of iterations **N** is reached
5. The model that produced the largest consensus set is returned



VIII. RANSAC: The Random Sample Consensus Algorithm

Algorithm

1. **Select a random subset of the original data. Call this subset the hypothetical inliers.**
2. Fit the model to the hypothetical inliers
3. Test all other data against the model and mark points either as inliers or outliers according to some loss function. The inliers are called “consensus set”
4. Return to step 1 until a predefined number of iterations **N** is reached
5. The model that produced the largest consensus set is returned

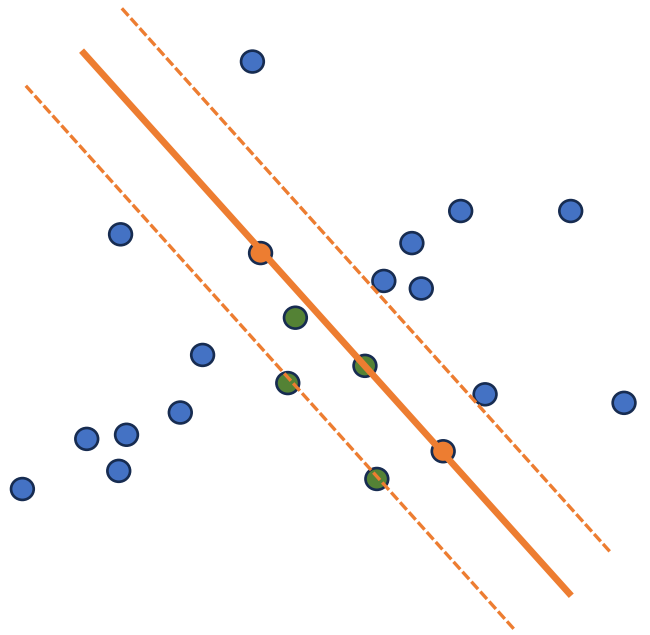


VIII. RANSAC: The Random Sample Consensus Algorithm

Algorithm

1. **Select a random subset of the original data. Call this subset the hypothetical inliers.**
2. Fit the model to the hypothetical inliers
3. Test all other data against the model and mark points either as inliers or outliers according to some loss function. The inliers are called “consensus set”
4. Return to step 1 until a predefined number of iterations **N** is reached
5. The model that produced the largest consensus set is returned

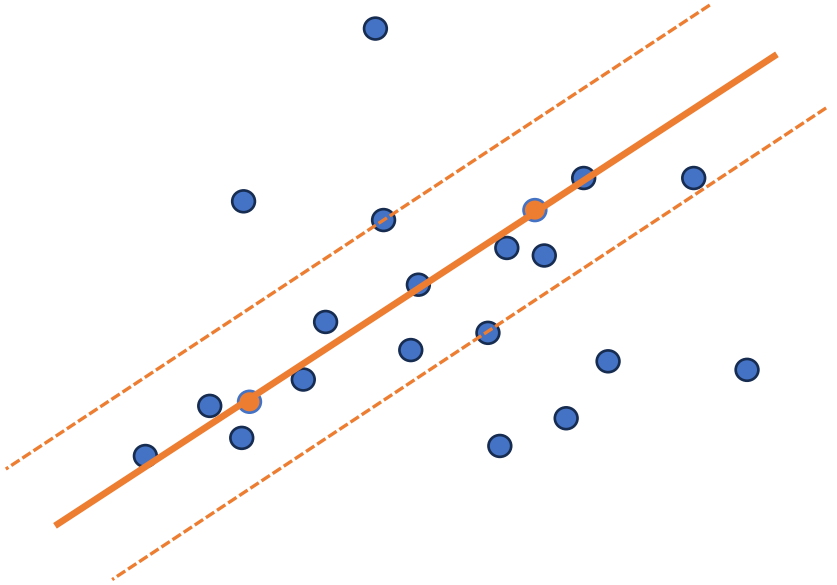
# of inliers = 4



VIII. RANSAC: The Random Sample Consensus Algorithm

Algorithm

1. Select a random subset of the original data. Call this subset the hypothetical inliers.
2. Fit the model to the hypothetical inliers
3. Test all other data against the model and mark points either as inliers or outliers according to some loss function. The inliers are called “consensus set”
4. Return to step 1 until a predefined number of iterations **N** is reached
5. The model that produced the largest consensus set is returned

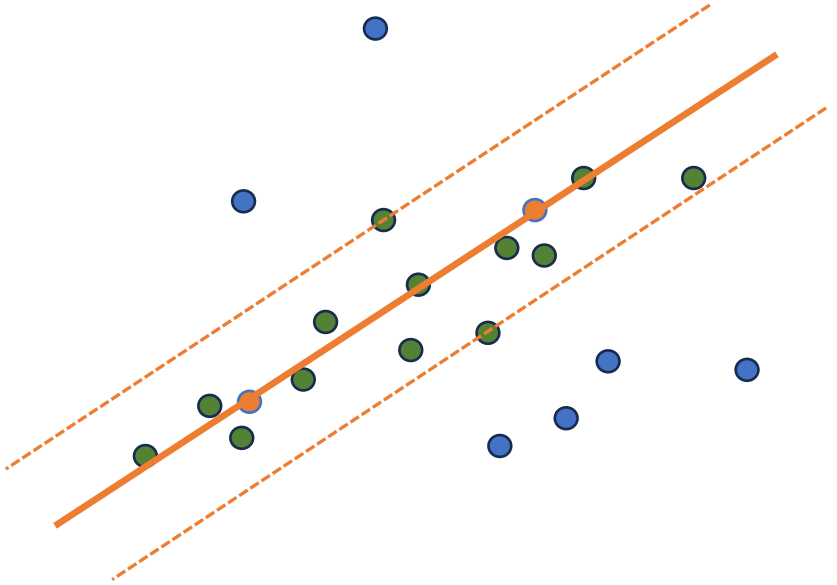


VIII. RANSAC: The Random Sample Consensus Algorithm

Algorithm

1. **Select a random subset of the original data. Call this subset the hypothetical inliers.**
2. Fit the model to the hypothetical inliers
3. Test all other data against the model and mark points either as inliers or outliers according to some loss function. The inliers are called “consensus set”
4. Return to step 1 until a predefined number of iterations **N** is reached
5. The model that produced the largest consensus set is returned

# of inliers = 13



VIII. RANSAC: The Random Sample Consensus Algorithm

Two questions:

1. Which possible final step can be added to the algorithm to improve the quality of the model?

2. How many iteration do we have to perfrom? In other words, how often do we need to try?

VIII. RANSAC: The Random Sample Consensus Algorithm

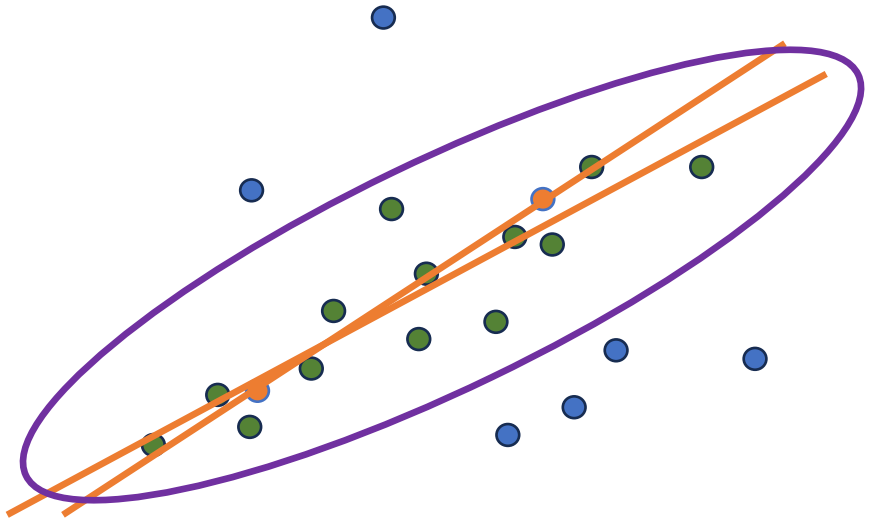
Algorithm

1. Select a random subset of the original data. Call this subset the hypothetical inliers.
2. Fit the model to the hypothetical inliers
3. Test all other data against the model and mark points either as inliers or outliers according to some loss function (a score). The inliers are called “consensus set”
4. Return to step 1 until a predefined number of iterations **N** is reached
5. The model that produced the largest consensus set is selected
6. Fit the model to the set of inliers associated with the selected model.

VIII. RANSAC: The Random Sample Consensus Algorithm

Algorithm

1. Select a random subset of the original data. Call this subset the hypothetical inliers.
2. Fit the model to the hypothetical inliers
3. Test all other data against the model and mark points either as inliers or outliers according to some loss function. The inliers are called “consensus set”
4. Return to step 1 until a predefined number of iterations **N** is reached
5. **The model that produced the largest consensus set is selected**
6. **Fit the model to the set of inliers associated with the selected model.**



# of inliers = 13

VIII. RANSAC: The Random Sample Consensus Algorithm

How many iterations?

s: minimum number of points needed to fit a model

e: Outlier ratio, i.e. probability that a point is an outlier (# outliers / total # of data points).

=> **N:** number of RANSAC interations (i.e. number of trials)

The idea is to choose **N** so that, **with a probability p**, at least one ramdom sample set is free of outliers.

$(1-e)$: Probability to draw one inlier.

$(1-e)^s$: Probability to draw ***s*** inliers.

$1-(1-e)^s$: Probability to fail **once**, i.e. to not draw only inliers.

$(1-(1-e)^s)^N$: Probability to fail **N times**.

$$(1-(1-e)^s)^N = 1-p$$

VIII. RANSAC: The Random Sample Consensus Algorithm

How many iterations?

s: minimum number of points needed to fit a model

e: Outlier ratio, i.e. probability that a point is an outlier (# outliers / total # of data points).

=> **N:** number of RANSAC interations (i.e. number of trials)

The idea is to choose **N** so that, **with a probability p**, at least one ramdom sample set is free of outliers.

$$(1-(1-e)^s)^N = 1-p$$

$$N\log(1-(1-e)^s) = log(1-p)$$

$$\boldsymbol{N = \frac{log(1-p)}{log(1-(1-e)^s)}}$$

VIII. RANSAC: The Random Sample Consensus Algorithm

How many iterations?

$$N = \frac{log(1-p)}{log(1-(1-e)^s)}$$

s: minimum number of points needed to fit a model

e: Outlier ratio.

p: probability of success

=> **N:** number of RANSAC interations (i.e. number of trials)

e	0,1						
	s						
p	**2**	**3**	**4**	**5**	**10**	**15**	**20**
0,1	1	1	1	1	1	1	1
0,5	1	1	1	1	2	4	6
0,75	1	2	2	2	4	7	11
0,9	2	2	3	3	6	10	18
0,95	2	3	3	4	7	13	24
0,99	3	4	5	6	11	20	36
0,999	5	6	7	8	17	30	54
0,9999	6	8	9	11	22	40	72

VIII. RANSAC: The Random Sample Consensus Algorithm

How many iterations?

$$N = \frac{log(1-p)}{log(1-(1-e)^s)}$$

s: minimum number of points needed to fit a model

e: Outlier ratio.

p: probability of success

=> **N:** number of RANSAC interations (i.e. number of trials)

e	0,3						
	s						
p	**2**	**3**	**4**	**5**	**10**	**15**	**20**
0,1	1	1	1	1	4	23	132
0,5	2	2	3	4	25	146	869
0,75	3	4	6	8	49	292	1 737
0,9	4	6	9	13	81	484	2 885
0,95	5	8	11	17	105	630	3 753
0,99	7	11	17	26	161	968	5 770
0,999	11	17	26	38	242	1 452	8 654
0,9999	14	22	34	51	322	1 936	11 539

VIII. RANSAC: The Random Sample Consensus Algorithm

How many iterations?

$$N = \frac{log(1-p)}{log(1-(1-e)^s)}$$

s: minimum number of points needed to fit a model

e: Outlier ratio.

p: probability of success

=> **N:** number of RANSAC interations (i.e. number of trials)

e	0,5						
	s						
p	**2**	**3**	**4**	**5**	**10**	**15**	**20**
0,1	1	1	2	4	108	3 453	110 479
0,5	3	6	11	22	710	22 713	726 818
0,75	5	11	22	44	1 419	45 426	1 453 635
0,9	9	18	36	73	2 357	75 450	2 414 435
0,95	11	23	47	95	3 067	98 163	3 141 252
0,99	17	35	72	146	4 714	150 900	4 828 869
0,999	25	52	108	218	7 071	226 350	7 243 303
0,9999	33	69	143	291	9 427	301 800	9 657 738

VIII. RANSAC: The Random Sample Consensus Algorithm

The minimum number of points *s* matters

- If for a problem, we have two algorithm that require different values of ***s***:

  s can be used as a decision criterion

  A smaller value of ***s*** => A more efficient RANSAC procedure.

RANSAC – Pros

- Robust to deal with outliers
- Works well with $s \in [1..10]$ (depending on *e*)
- Easy to understand and to implement

RANSAC – Cons

- Computational times grows quickly with *e* and *s*.
- Not good for getting multiple fits.

VIII. RANSAC: The Random Sample Consensus Algorithm

Applications

- Robust Line and Plane Fitting

  Fit lines in 2D, planes in 3D, even when many points are outliers.

- Homography Estimation

  Find projective transformations between two images.

  Key in panorama stitching, augmented reality, image registration.

- Model Fitting in Object Recognition

  Fit shapes (lines, circles, ellipses, 3D models) to detected features.

  Recognize partially occluded or noisy objects.

- Pose Estimation (Perspective-n-Point – PnP – problem)

  Estimate a camera's position and orientation from known 3D-2D point correspondences.

  Used in robotics, AR, self-driving cars.

- …